

UNIVERSIDADE NOVA DE LISBOA

Faculdade de Ciências e Tecnologia

Licenciatura em
Engenharia Eletrotécnica e de Computadores

Programa de Introdução à Investigação Científica em
Engenharia Eletrotécnica e de Computadores

Relatório de Atividade
Network Intrusion Detection
System (NIDS)

Diogo Neto Filipe

66434

Relatório apresentado na Faculdade de Ciências e Tecnologia da Universidade Nova de Lisboa para avaliação no Programa de Introdução à Investigação Científica em Engenharia Eletrotécnica e de Computadores

Luís Bernardo, DEEC

Fevereiro, 2026

Resumo

Com o crescimento exponencial das redes de computadores e da dependência global do meio digital, a cybersegurança tornou-se um dos pilares fundamentais da sociedade moderna. A detecção precoce de atividades maliciosas é essencial para proteger sistemas, dados sensíveis e serviços essenciais. Neste contexto, os Sistemas de Detecção de Intrusões em Rede (Network Intrusion Detection Systems NIDS) desempenha um papel central na identificação automática de padrões anómalos ou comportamentos associados a ataques.

Nos últimos anos as técnicas de Machine Learning (ML) e Deep Learning (DL) têm sido amplamente exploradas como alternativa às abordagens tradicionais. Estas técnicas permitem modelar padrões complexos de tráfego de rede e adaptar-se a novas ameaças, oferecendo maior capacidade de generalização. No entanto, a comparação sistemática entre modelos clássicos de aprendizagem automática e arquiteturas de aprendizagem profunda continua a ser um tema relevante de investigação.

Este projeto propõe a implementação e avaliação experimental de diferentes modelos de classificação aplicados à detecção de intrusões, utilizando os conjuntos de dados KDD-Cup99 e NSL-KDD. Serão analisados modelos clássicos, como Regressão Logística, Naive Bayes, K-Nearest Neighbors e Random Forest, bem como arquiteturas baseadas em redes neurais convolucionais. A avaliação será realizada com base em métricas como accuracy, precision, recall e F1-score, permitindo uma análise comparativa rigorosa do desempenho das abordagens estudadas.

O objetivo final é compreender as vantagens, limitações e aplicabilidade prática de cada paradigma no contexto de sistemas modernos de detecção de intrusões.

Abstract

With the exponential growth of computer networks and the global dependence on digital infrastructures, cybersecurity has become one of the fundamental pillars of modern society. The early detection of malicious activities is essential to protect systems, sensitive data, and critical services. In this context, Network Intrusion Detection Systems (NIDS) play a central role in the automatic identification of anomalous patterns or behaviors associated with cyberattacks.

In recent years, Machine Learning (ML) and Deep Learning (DL) techniques have been widely explored as alternatives to traditional approaches. These techniques enable the modeling of complex network traffic patterns and allow systems to adapt to new threats, offering greater generalization capabilities. However, the systematic comparison between classical machine learning models and deep learning architectures remains a relevant research topic.

This project proposes the implementation and experimental evaluation of different classification models applied to intrusion detection, using the KDD-Cup99 and NSL-KDD datasets. Classical models such as Logistic Regression, Naive Bayes, K-Nearest Neighbors, and Random Forest will be analyzed, as well as architectures based on Convolutional Neural Networks. The evaluation will be conducted using metrics such as accuracy, precision, recall, and F1-score, enabling a rigorous comparative analysis of the performance of the studied approaches.

The final objective is to understand the advantages, limitations, and practical applicability of each paradigm in the context of modern intrusion detection systems.

Conteúdo

Resumo	2
Abstract	3
1 Introdução	1
1.1 Contexto	1
1.2 Atividades desenvolvidas no âmbito do PIIC	1
2 Datasets	3
2.1 KDDCup99	3
2.2 NSL-KDD	4
2.3 Labels	4
2.3.1 Labels	4
3 Metodologia	5
3.1 Pipeline	5
3.2 Configuração Experimental	5
3.2.1 Modo Binary	5
3.2.2 Modo Multiclass	5
3.2.3 Modo One-vs-Rest (OvR)	6
3.3 Métricas	6
4 Resultados	7
4.1 Resultados — Binary	7
4.2 Resultados — Multiclass	8
4.3 Resultados — Deep Learning (CNN)	10
5 Conclusões	12

Lista de Figuras

4.1	Distribuição de classes no treino (Baseline)	7
4.2	Distribuição das famílias no conjunto de teste (Multiclass).	9
4.3	Matriz de confusão do modelo CNN no cenário Binary.	10

Lista de Tabelas

4.1	Distribuição de classes no treino (Baseline)	7
4.2	Melhor resultado (GLOBAL) — K-Nearest Neighbors	8
4.3	Pior resultado (GLOBAL) — SVM (GridSearchCV)	8
4.4	Acertos/erros por família: melhor (Decision Tree) vs pior (Logistic Regression).	10

Capítulo 1

Introdução

1.1 Contexto

O Programa de Introdução à Prática Profissional (PIPP), promovido pelo Departamento de Engenharia Eletrotécnica e de Computadores da NOVA School of Science and Technology, tem como objetivo proporcionar aos estudantes um primeiro contacto estruturado com um contexto de investigação na área da sua formação. O programa pretende aproximar a formação académica da prática real, promovendo o desenvolvimento de competências técnicas, organizacionais e comportamentais essenciais para o exercício futuro da profissão. O projeto centrou-se na implementação e avaliação experimental de modelos de Machine Learning e Deep Learning aplicados à deteção de intrusões em redes de computadores (Network Intrusion Detection Systems – NIDS). Foram utilizados conjuntos de datasets de referência, nomeadamente KDD-Cup99 e NSL-KDD, com o objetivo de comparar o desempenho de diferentes abordagens de classificação na identificação de tráfego malicioso.

O presente relatório encontra-se estruturado em cinco secções principais e um anexo. Após esta Introdução, é apresentado o enquadramento do projeto e os seus objetivos técnicos, seguindo-se a descrição das atividades desenvolvidas, das metodologias utilizadas e do impacto do trabalho na minha formação académica e profissional. O relatório termina com uma reflexão crítica sobre a experiência adquirida, sendo incluído em anexo material complementar relevante.

1.2 Atividades desenvolvidas no âmbito do PIIC

No âmbito do PIIC, desenvolvi um projeto na área da cibersegurança focado na implementação e avaliação experimental de modelos de classificação aplicados à deteção de intrusões em redes de computadores (NIDS). O trabalho incluiu o estudo e pré-processamento dos conjuntos de dados KDD-Cup99 e NSL-KDD, a conversão dos ficheiros originais para formatos adequados a treino supervisionado, a codificação de variáveis categóricas, a normalização das variáveis numéricas e a divisão dos dados em conjuntos de treino e teste. Procedi à implementação de modelos clássicos de Machine Learning, nomeadamente Regressão Logística, Naive Bayes, K-Nearest Neighbors e Random Forest, em versões multi-class e One-vs-Rest, recorrendo à biblioteca scikit-learn. Desenvolvi igualmente arquiteturas baseadas em redes neuronais convolucionais (CNN) utilizando TensorFlow/Keras, definindo as camadas convolucionais, de pooling, dropout e fully connected, e realizando o treino em GPU. Efetuei a avaliação sistemática dos modelos através do cálculo de mé-

tricas como accuracy, precision, recall e F1-score, bem como da análise de matrizes de confusão e do impacto do desbalanceamento das classes. Organizei o código de forma modular, implementei mecanismos de registo automático de resultados experimentais e assegurei a reprodutibilidade dos testes através da fixação de sementes aleatórias.

Capítulo 2

Datasets

2.1 KDDCup99

O KDDCup99 é um conjunto de dados amplamente utilizado na investigação em deteção de intrusões em redes de computadores (Network Intrusion Detection Systems – NIDS). Foi construído a partir do programa de avaliação de deteção de intrusões promovido pelo MIT Lincoln Laboratory no âmbito da iniciativa DARPA de 1998, com o objetivo de disponibilizar um dataset padronizado que permitisse avaliar e comparar diferentes abordagens de deteção de ataques. Os dados resultam da captura de nove semanas de tráfego TCP numa rede local que simulava um ambiente militar real, contendo tráfego legítimo e múltiplos ataques injetados de forma controlada. O raw TCP dump foi processado e convertido em registos de conexões, sendo cada conexão definida como uma sequência de pacotes TCP entre um endereço IP de origem e um endereço IP de destino, sob um determinado protocolo, com início e fim bem definidos. Após processamento, foram obtidos cerca de cinco milhões de registos para treino e aproximadamente dois milhões para teste. Cada registo é apresentado sob a forma de uma linha num ficheiro estruturado, onde cada coluna corresponde a um atributo da conexão e a última coluna representa o rótulo (classe). No total, cada conexão é descrita por 41 atributos, que podem ser contínuos ou discretos, e é classificada como “normal” ou como um tipo específico de ataque. Os ataques estão organizados em quatro categorias principais: Denial of Service (DoS), Remote to Local (R2L), User to Root (U2R) e Probing. Uma característica importante do dataset é que o conjunto de teste não segue exatamente a mesma distribuição do conjunto de treino e inclui tipos de ataques adicionais, o que torna a tarefa de generalização mais exigente. Apesar da sua ampla utilização, o KDDCup99 apresenta limitações conhecidas, nomeadamente forte desbalanceamento entre classes e redundância de registos, aspetos que devem ser considerados na avaliação de modelos de aprendizagem automática. Os dados são divididos então em 3 grandes grupos:

- **Basic features** onde são expostas as características básicas de uma ligação TCP
- **Content features** baseadas no conteúdo da conexão e conhecimento do domínio
- **Traffic features** baseadas em padrões temporais

Alguns dos pontos mais importantes a referenciar são:

- **count:** — Número de conexões realizadas para o mesmo host de destino nos dois segundos anteriores, permitindo detetar comportamentos de tráfego intensivo associados a ataques DoS.

- **error-rate:** — Percentagem de conexões para o mesmo host que apresentam erros do tipo SYN, indicando possíveis tentativas massivas de abertura de ligações não concluídas.
- **srv-count:** — Número de conexões para o mesmo serviço nos dois segundos anteriores, permitindo identificar padrões repetitivos direcionados a um serviço específico.
- **srv-error-rate:** Percentagem de conexões para o mesmo serviço que apresentam erros SYN, sendo útil para detetar ataques que exploram vulnerabilidades associadas a um serviço particular.

Estas características foram selecionadas por representarem métricas de tráfego temporal diretamente relacionadas com padrões típicos de ataques de negação de serviço e sondagem, permitindo capturar comportamentos anómalos que não seriam identificáveis apenas através de atributos individuais da conexão

2.2 NSL-KDD

O NSL-KDD é uma versão melhorada do KDDCup99, criada com o objetivo de corrigir algumas das suas limitações, nomeadamente a elevada redundância de registos e o forte desbalanceamento entre classes. Ao remover exemplos duplicados e reduzir o número de instâncias repetidas, o NSL-KDD proporciona uma avaliação mais equilibrada e realista dos modelos de Machine Learning, evitando que classificadores obtenham resultados artificialmente elevados devido à repetição excessiva de padrões semelhantes. Mantém a mesma estrutura de 41 atributos + 1 coluna de dificuldade -e as mesmas categorias de ataques, permitindo comparações diretas com trabalhos baseados no KDD99.

2.3 Labels

2.3.1 Labels

Com o objetivo de realizar uma investigação, para além de ter utilizado vários modelos de Machine Learning, utilizei também várias estruturas para comparação de dados de entre elas:

- **Binary:** normal vs attack
- **MultiClass:** attack-type (ex: neptune, ipsweep, etc.)
- **MultiClass OVR** onde foi comparado e ensinado cada ataque individualmente

Capítulo 3

Metodologia

3.1 Pipeline

A estrutura do trabalho foi organizada de forma modular, seguindo princípios de boa prática de programação adquiridos ao longo do curso, nomeadamente separação de responsabilidades, reutilização de código e reprodutibilidade experimental.

O pipeline inicia-se com a fase de carregamento dos dados, onde é selecionado o ficheiro correspondente ao conjunto de treino e ao conjunto de teste. Segue-se a etapa de pré-processamento, na qual os dados são convertidos para um formato adequado aos algoritmos de aprendizagem automática. Nesta fase, as variáveis categóricas do dataset (como *protocol-type*, *service* e *flag*) são codificadas através de mapeamento para valores numéricos, utilizando estruturas do tipo dicionário, permitindo que os modelos possam processá-las corretamente.

Após a conversão, os dados são normalizados e estruturados em matrizes de características e vetores de rótulos, ficando preparados para o treino e posterior avaliação dos modelos. Por fim, o sistema executa o treino, realiza as previsões sobre o conjunto de teste e apresenta os resultados através das métricas de desempenho definidas.

3.2 Configuração Experimental

3.2.1 Modo Binary

Nesta configuração, o problema é formulado como uma tarefa de classificação binária, onde o objetivo consiste em distinguir entre ligações *normais* e *ataques*. Todas as categorias de ataque são agregadas numa única classe, sendo o modelo treinado apenas para aprender a diferença entre comportamento legítimo e comportamento malicioso. Desta forma, o classificador não diferencia tipos específicos de ataque, mas aprende um padrão geral de tráfego normal, classificando como anómala qualquer ligação que se afaste desse padrão. Esta abordagem simplifica o problema e permite avaliar a capacidade do modelo em detetar intrusões de forma global..

3.2.2 Modo Multiclass

No modo *Multiclass*, o problema é formulado como uma tarefa de classificação com múltiplas classes, onde o modelo aprende a distinguir entre todas as categorias presentes no

dataset. Neste caso, cada tipo de ataque é tratado como uma classe distinta, juntamente com a classe *normal*.

O classificador não se limita a identificar se uma ligação é maliciosa, mas procura determinar especificamente qual o tipo de ataque associado. Esta abordagem é mais exigente, uma vez que obriga o modelo a aprender padrões diferenciados para cada categoria, aumentando a complexidade do problema de classificação.

3.2.3 Modo One-vs-Rest (OvR)

No modo *One-vs-Rest* (OvR), o problema multiclass é decomposto em vários problemas de classificação binária independentes. Para cada classe existente no dataset, é treinado um classificador que distingue essa classe específica de todas as restantes.

Assim, para um conjunto com N classes, são treinados N classificadores binários, onde cada modelo aprende a responder à questão: “esta ligação pertence à classe X ou não?”. Durante a fase de previsão, o modelo que apresentar a maior confiança determina a classe final atribuída à conexão.

Esta abordagem permite tratar o problema multiclass utilizando algoritmos originalmente concebidos para classificação binária, mantendo uma estrutura modular e facilitando a análise individual do desempenho por classe.

3.3 Métricas

As métricas de avaliação utilizadas baseiam-se na matriz de confusão, considerando:

- TP (True Positives): número de ataques corretamente identificados;
- TN (True Negatives): número de ligações normais corretamente classificadas;
- FP (False Positives): número de ligações normais incorretamente classificadas como ataque;
- FN (False Negatives): número de ataques não detetados.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad F1 = \frac{2PR}{P + R}$$

Capítulo 4

Resultados

4.1 Resultados — Binary

A Tabela 4.1 e a Figura 4.1 apresentam a distribuição das classes no conjunto de treino. Verifica-se uma distribuição relativamente equilibrada entre ligações *normais* (53.46%) e *ataques* (46.54%), não existindo um desbalanceamento extremo que pudesse enviesar significativamente o processo de treino dos classificadores.

Tabela 4.1: Distribuição de classes no treino (Baseline)

Classe	Nº amostras	Percentagem
Normal (0)	67343	53.46%
Attack (1)	58630	46.54%

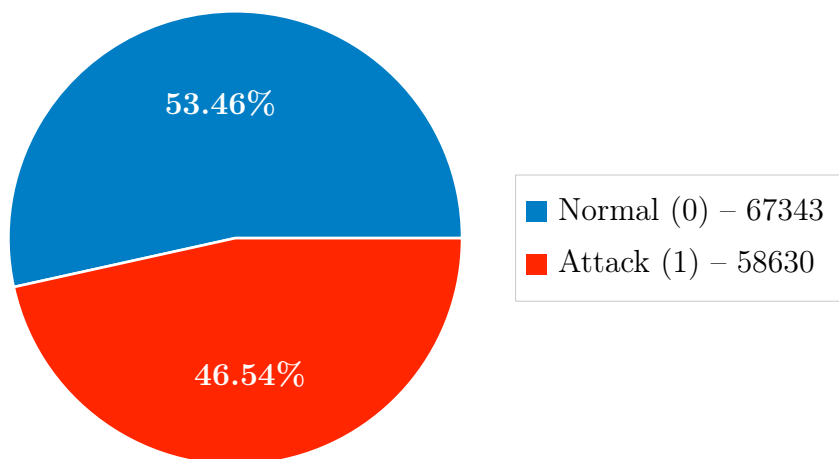


Figura 4.1: Distribuição de classes no treino (Baseline)

Foram avaliados vários modelos clássicos de Machine Learning no cenário de classificação binária. A comparação foi realizada com base nas métricas definidas anteriormente, dando especial relevância ao *F1-score*, por representar um equilíbrio entre *Precision* e *Recall*, sendo particularmente adequado em problemas de deteção de intrusões.

O melhor desempenho global foi obtido pelo modelo **K-Nearest Neighbors (KNN)**, conforme apresentado na Tabela 4.2. Este modelo apresenta uma *Precision* elevada

(0.9675), indicando uma baixa taxa de falsos positivos, e um *Recall* de 0.6510, demonstrando uma capacidade consistente de detecção de ataques. O *F1-score* obtido foi de 0.7783, o mais elevado entre os modelos avaliados. Adicionalmente, a *False Positive Rate* reduzida (0.0289) torna este modelo particularmente interessante no contexto de NIDS, onde alarmes excessivos podem comprometer a eficácia operacional do sistema.

Tabela 4.2: Melhor resultado (GLOBAL) — K-Nearest Neighbors

Métrica	Valor
TN	9430.0000
FP	281.0000
FN	4479.0000
TP	8354.0000
Accuracy	0.7889
Precision	0.9675
Recall	0.6510
F1-score	0.7783
TPR	0.6510
FPR	0.0289

Em contraste, o pior desempenho global foi registrado pelo modelo **SVM** (otimizado via *GridSearchCV*), apresentado na Tabela 4.3. Apesar de manter uma *Precision* elevada (0.9024), o modelo apresentou um *Recall* inferior (0.5035), indicando uma maior quantidade de ataques não detetados (valores de FN mais elevados). Como consequência, o *F1-score* obtido foi de 0.6463, inferior ao do melhor modelo. Verifica-se igualmente uma *False Positive Rate* superior (0.0720), evidenciando um menor equilíbrio entre sensibilidade e especificidade.

Tabela 4.3: Pior resultado (GLOBAL) — SVM (GridSearchCV)

Métrica	Valor
TN	9012.0000
FP	699.0000
FN	6372.0000
TP	6461.0000
Accuracy	0.6863
Precision	0.9024
Recall	0.5035
F1-score	0.6463
TPR	0.5035
FPR	0.0720

4.2 Resultados — Multiclass

Nesta configuração, o problema é formulado como classificação multiclasse, onde cada ligação é atribuída a uma classe específica (*normal* ou um tipo de ataque). Para além das

métricas globais, foi também analisado o desempenho por *família* de ataque (DoS, Probe, R2L, U2R e *unknown*), de forma a compreender melhor onde os modelos falham.

A Figura 4.2 apresenta a distribuição das famílias no conjunto de teste. Observa-se que as classes *normal* e *dos* representam a maioria das amostras, enquanto as famílias *r2l* e *u2r* são extremamente minoritárias. Esta assimetria contribui para que modelos obtenham *accuracy* relativamente elevada, mesmo quando falham em classes raras, penalizando significativamente as métricas macro (macro *precision*, macro *recall* e macro *F1-score*).

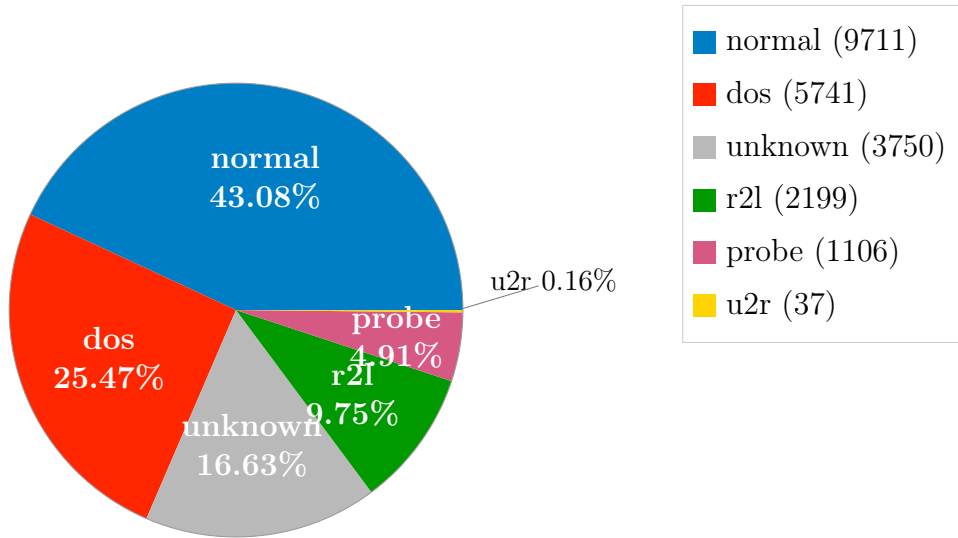


Figura 4.2: Distribuição das famílias no conjunto de teste (Multiclass).

De entre os modelos avaliados, o melhor desempenho global (critério: macro *F1-score*) foi obtido pela **Decision Tree**, com *Accuracy* = 0.7143 e macro *F1-score* = 0.2644. Em contraste, o pior desempenho global foi registado pela **Logistic Regression**, com *Accuracy* = 0.6620 e macro *F1-score* = 0.1000. Apesar de a *accuracy* ser relativamente elevada em ambos os casos, as métricas macro evidenciam que a performance está longe de ser uniforme entre classes.

A Tabela 4.4 compara os acertos por família para o melhor e o pior modelo. Observa-se que ambos os modelos mantêm um desempenho elevado em *normal* (acima de 97%), refletindo a predominância desta classe e a separabilidade relativamente fácil face a algumas categorias. No entanto, as diferenças tornam-se claras em famílias específicas: a **Decision Tree** apresenta melhorias substanciais em *probe* (97.11% vs 31.83%) e em *dos* (97.53% vs 89.11%), enquanto a **Logistic Regression** falha amplamente na deteção de ataques menos lineares ou com padrões mais heterogêneos.

Adicionalmente, verifica-se que a família *unknown* tem 0% de acerto em ambos os modelos, indicando ausência de capacidade de generalização para essas classes no cenário testado. As famílias *r2l* e *u2r*, por serem raras e tipicamente mais difíceis (e.g., comportamentos semelhantes a tráfego legítimo), apresentam também desempenho muito baixo, sendo particularmente penalizadas nas métricas macro e contribuindo para os valores reduzidos de macro *precision*, macro *recall* e macro *F1-score*.

Tabela 4.4: Acertos/erros por família: melhor (Decision Tree) vs pior (Logistic Regression).

Família	Total	Decision Tree (melhor)			Logistic Regression (pior)		
		Correct	Wrong	Acc (%)	Correct	Wrong	Acc (%)
normal	9711	9419	292	96.99	9457	254	97.38
dos	5741	5599	142	97.53	5116	625	89.11
unknown	3750	0	3750	0.00	0	3750	0.00
r2l	2199	2	2197	0.09	0	2199	0.00
probe	1106	1074	32	97.11	352	754	31.83
u2r	37	10	27	27.03	0	37	0.00

4.3 Resultados — Deep Learning (CNN)

Para superar as limitações dos modelos lineares, implementou-se uma rede neuronal convolucional (CNN). Embora estas redes sejam tipicamente associadas ao processamento de imagem, foram aqui aplicadas sobre os vetores de atributos unidimensionais (1D) para capturar correlações locais entre as características do tráfego.

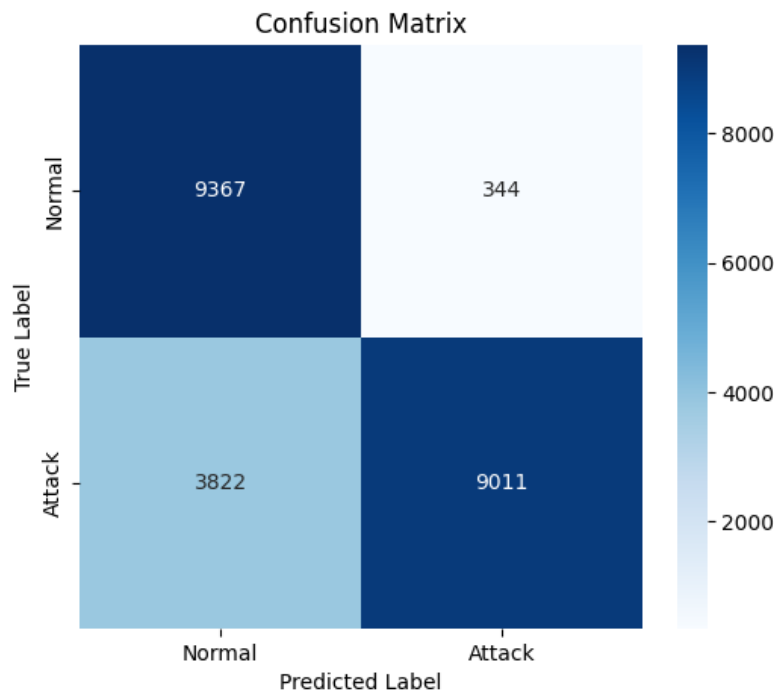


Figura 4.3: Matriz de confusão do modelo CNN no cenário Binary.

A arquitetura CNN demonstrou ser a mais eficaz no cenário binário. Os resultados obtidos no conjunto de teste revelam um desempenho superior aos modelos clássicos:

- **Accuracy:** 0.8200
- **Precision (Attack):** 0.9600

- **Recall (Attack):** 0.7000
- **F1-score (Attack):** 0.8100

Ao compararmos com o melhor modelo de Machine Learning clássico (K-Nearest Neighbors, $F1 = 0.7783$), a CNN registou uma melhoria de aproximadamente 3% no F1-score. Este ganho deve-se, sobretudo, ao aumento da taxa de deteção (*Recall*), que passou de 0.6510 para 0.7000.

Na prática, isto significa que a rede neuronal foi capaz de identificar 3822 ataques que modelos como o SVM ou Naive Bayes tenderiam a classificar incorretamente como tráfego normal. Este resultado reforça a premissa de que a extração automática de características por camadas convolucionais é mais robusta na modelação de padrões complexos de cibersegurança do que os algoritmos baseados em distâncias ou probabilidades simples.

Capítulo 5

Conclusões

O presente trabalho teve como objetivo a implementação e avaliação experimental de diferentes abordagens de Machine Learning e Deep Learning aplicadas à detecção de intrusões em redes de computadores, recorrendo aos datasets KDDCup99 e NSL-KDD.

No cenário de classificação binária (Normal vs Attack), os resultados demonstraram que a arquitetura baseada em Convolutional Neural Networks (CNN) apresentou o melhor desempenho global, superando os modelos clássicos em termos de *accuracy* e *F1-score*. Destaca-se particularmente a melhoria na taxa de detecção de ataques (*Recall*), evidenciando maior capacidade de modelação de padrões complexos presentes no tráfego de rede.

No entanto, a análise detalhada revelou que, embora o desempenho global seja elevado, continuam a existir falsos negativos relevantes, o que demonstra que mesmo modelos baseados em Deep Learning não garantem detecção perfeita de intrusões.

No cenário multiclass, verificou-se que a detecção de ataques pertencentes às famílias DoS e Probe apresenta resultados significativamente superiores quando comparada com ataques das categorias R2L e U2R. Esta discrepância está fortemente associada ao desbalanceamento do dataset e à escassez de exemplos em determinadas classes, evidenciando uma limitação estrutural dos dados utilizados.

De forma geral, conclui-se que:

- Modelos clássicos apresentam desempenho competitivo em cenários binários;
- Arquiteturas baseadas em Deep Learning demonstram melhor capacidade de generalização;
- O desbalanceamento das classes constitui um fator crítico no desempenho multiclass;
- A escolha do modelo deve considerar o compromisso entre precisão e capacidade de detecção de ataques.

Este trabalho permitiu não apenas comparar diferentes paradigmas de aprendizagem automática, mas também compreender as limitações práticas associadas à aplicação de técnicas de classificação em contextos reais de cibersegurança.